

OCI Cloud Cost Optimization Tool

Automated analytics, reporting & scheduled delivery

Project brief and phased Claude Code implementation prompts

Document purpose

This document captures the full project brief, scope, technical context, and the five-phase Claude Code prompt suite for building the OCI Cloud Cost Optimization tool. It is intended as the single source of truth for internal review before implementation begins.

PREPARED

07 May 2026

AUDIENCE

Internal engineering team

STATUS

Draft for review

VERSION

1.0

Executive summary

We are building a lightweight, Python-based tool that runs every 15 days, pulls compute metrics and cost data from Oracle Cloud Infrastructure (OCI), identifies over-provisioned resources, performs detailed cost-utilization analytics, and delivers a formatted PDF report directly to the team via email. The tool is designed to run on Linux and Windows, with zero manual effort once configured, enabling proactive cloud-spend reduction on an ongoing basis.

CADENCE	PLATFORMS	OUTPUT	GOAL
Every 15 days	Linux + Windows	PDF via SMTP	Cut OCI spend

Why this matters

Cloud spend tends to drift upward silently. Instances are provisioned for peak-load assumptions that rarely materialize, volumes get attached and forgotten, and shapes selected six months ago are no longer the right fit. Without automated correlation between resource utilization and cost, these inefficiencies compound. This tool correlates the two on a fixed cadence and surfaces actionable, ranked recommendations so the team can make better cost decisions with confidence.

Outcomes

- **Reduced cloud spend** through identification of over-provisioned, idle, and orphaned resources with concrete, ranked savings estimates.
- **Zero manual overhead** via cron (Linux) / Task Scheduler (Windows), with email delivery to a configured distribution list.
- **Defensible, CFO-grade analytics** with confidence scoring, utilization pattern classification, and statistical anomaly detection.
- **Operational reliability** through structured logging, health checks, idempotent steps, and resumable runs.
- **Auditability** via raw-data archival, JSON analytics output, and append-only run logs.

Project context & requirements

Original brief

AS DESCRIBED BY THE REQUESTER

A Python project that uses the OCI library to fetch all data regarding resource utilization and cost per machine, derives analytics into a PDF report (under-utilization, over-commitment, important ratios, and other derivable insights), and emails it on a schedule via SMTP. The application must be Linux- and Windows-compatible, runnable as a cron job or scheduled task, and perform the full pipeline — collection, processing, report generation, analytics derivation, and email delivery — without manual intervention. The purpose is to correlate resource utilization with cost so the team can make better cost-saving decisions.

Scope — what the tool does

The tool covers the end-to-end loop from OCI data collection through analytics, report generation, and notification, on a fixed 15-day cadence:

- **Collection.** Pulls compute instance metadata, fine-grained metrics (CPU, memory, network, disk), block-volume details, and itemized cost data from OCI APIs across all configured compartments and regions.
- **Analytics.** Profiles utilization (mean, percentiles, pattern classification), attributes cost per resource, derives KPI ratios (cost per vCPU, cost per utilization point, wasted spend), generates right-sizing recommendations with confidence scoring, and detects anomalies.
- **Reporting.** Produces a multi-page, print-ready PDF with cover, executive summary, charts, instance-detail tables, recommendations, anomalies, and methodology.
- **Delivery.** Sends the PDF via SMTP (STARTTLS or SSL) to a configured distribution list, with retries, partial-success handling, and a test mode that writes RFC-compliant .eml files for offline review.
- **Scheduling.** Installs a cron job (Linux/macOS) or a Task Scheduler task (Windows) via a single command; idempotent, with status and uninstall sub-commands.

Non-goals

- Real-time monitoring or alerting (this is a 15-day batch tool).
- Multi-cloud (AWS, Azure, GCP) — OCI only in v1.
- Automated remediation — recommendations only; humans approve and execute.
- Web UI — CLI and email are the only interfaces.

Compatibility & runtime

Concern	Decision
Language	Python 3.9 or higher
Operating systems	Ubuntu 22.04, RHEL 8, Windows Server 2022, Windows 11
OCI SDK	oci-python-sdk (config-file auth + instance principals)
Scheduling	cron on Linux/macOS, Task Scheduler on Windows
Distribution	Source + venv install scripts; optional Docker image
Secrets	Environment variables; never committed to YAML

Architecture overview

The system is structured as a set of clearly-scoped modules connected by a thin orchestrator. Each stage produces a typed, serializable artifact so any single phase can be re-run, debugged, or replaced independently.

Pipeline stages

Stage	Module	Output artifact
1. Collect	src/collector/	reports/raw/<ts>_raw.json
2. Analyze	src/analytics/	reports/analytics/<ts>_analytics.json
3. Report	src/reporter/	reports/OCI_Cost_Report_<ts>.pdf
4. Notify	src/notifier/	SMTP delivery + .eml in test mode
5. Schedule	scripts/setup_schedule.py	cron entry / Task Scheduler XML

Project layout

```
oci-cost-optimizer/
├── config/
│   ├── config.yaml # all user-configurable settings
│   ├── shapes.json # OCI shape catalog with pricing
│   └── .env.example # secrets template
├── src/
│   ├── collector/ # OCI SDK clients, metric & cost fetchers
│   ├── analytics/ # utilization, cost mapping, right-sizing,
│   │   └── # ratios, anomaly detection
│   ├── reporter/ # ReportLab + matplotlib PDF generator
│   ├── notifier/ # SMTP delivery + optional Slack/Teams
│   ├── orchestrator/ # PipelineRunner, Step abstractions, state
│   ├── models/ # dataclasses & pydantic schemas
│   └── utils/ # logging, redaction, dates, health, metrics
├── scripts/
│   ├── setup_schedule.py # cross-platform scheduler installer
│   ├── install.sh / .ps1 # bootstrap scripts
│   └── uninstall.sh / .ps1
├── tests/ # unit, integration, performance, chaos
├── docs/ # ARCHITECTURE, RUNBOOK, SECURITY,
│   └── # CONFIGURATION
├── main.py # CLI entry point
├── pyproject.toml
├── requirements.txt
├── Dockerfile
└── README.md
```

Key design choices

- **Typed dataclasses end-to-end.** No untyped dicts crossing module boundaries — every artifact has a schema, validated on load.
- **Decimal for currency.** Floats lie about money; Decimal does not.

- **Pattern-aware right-sizing.** A bursty instance at 15% average CPU is not the same as a steady one — recommendation safety multipliers vary by detected pattern.
- **Confidence scoring on every recommendation.** Operators need to know which suggestions are high-confidence (\$-saving) versus low-confidence (worth a closer look).
- **Idempotent, resumable steps.** A failed run can be resumed from the last completed step, reusing artifacts.
- **Secrets never on disk.** Sensitive values come from env vars, OS keychain, or instance principals — never from config.yaml.
- **Cross-platform without platform-specific code paths leaking.** All filesystem operations use pathlib; the scheduler abstracts cron vs Task Scheduler behind a uniform CLI.

Phase roadmap

The build is split into five discrete, independently testable phases. Each phase ends with a green test suite and a working CLI command, so the team can review at every checkpoint before proceeding.

1	Project scaffold & OCI data collection Project structure, OCI SDK auth, config management, validated data collection from Monitoring and Usage APIs.
2	Analytics engine Statistical utilization profiling, pattern classification, cost attribution, right-sizing with confidence scoring, fleet KPIs, and multi-signal anomaly detection.
3	PDF report generator Multi-page branded PDF with auto-narrative, executive summary, charts (treemap, scatter, distribution, trend), per-instance tables, prioritized recommendations, anomalies, methodology.
4	Delivery, scheduling & orchestration SMTP delivery with retries and partial-success handling, optional Slack/Teams channels, full pipeline orchestrator with resumable steps, cross-platform scheduler installer.
5	Hardening, security, observability & release Secrets redaction, dependency scans, structured logging, metrics, chaos & performance tests, Docker packaging, runbooks, and final cross-platform validation.

HOW TO USE THESE PROMPTS

Each phase is a self-contained Claude Code prompt. Start a fresh session for each phase. From Phase 2 onward, prepend a short context bridge: "Phase N is complete. Here is the project tree: [paste output of `tree src/`]." This gives Claude Code the structure it needs without re-reading earlier work. Each phase ends with a verifiable definition of done — green tests, working CLI commands — so progress is reviewable at every step.

Implementation prompts

The five prompts that follow are written in the voice of a senior engineer briefing Claude Code. They are intentionally explicit about requirements, edge cases, testing expectations, and definitions of done. Treat them as engineering specifications first and prompts second.

PHASE 1

Project scaffold & OCI data collection

Foundation — typed collectors with retry, pagination, multi-region

You are a senior Python cloud engineer building a production-grade OCI Cloud Cost Optimization tool. Your task in this phase is to create the complete project scaffold and implement all OCI data collection.

=== PROJECT STRUCTURE ===

Create the following layout:

```
oci-cost-optimizer/
  ■■■ config/
  ■ ■■■ config.yaml # all user-configurable settings
  ■ ■■■ .env.example # env var template (never commit secrets)
  ■■■ src/
  ■ ■■■ __init__.py
  ■ ■■■ collector/
  ■ ■ ■■■ __init__.py
  ■ ■ ■■■ oci_client.py # OCI SDK client factory & auth
  ■ ■ ■■■ compute.py # fetch compute instance metrics
  ■ ■ ■■■ cost.py # fetch cost/usage reports
  ■ ■ ■■■ storage.py # fetch block volume & object storage metrics
  ■ ■■■ models/
  ■ ■ ■■■ __init__.py
  ■ ■ ■■■ schemas.py # dataclasses for Instance, Metric, CostRecord
  ■ ■■■ utils/
  ■ ■■■ __init__.py
  ■ ■■■ logger.py # structured logging (JSON + human-readable)
  ■ ■■■ date_utils.py # period helpers (last 15 days, etc.)
  ■■■ tests/
  ■ ■■■ test_collector/
  ■ ■■■ test_oci_client.py
  ■■■ main.py # CLI entry point
  ■■■ requirements.txt
  ■■■ pyproject.toml
  ■■■ README.md
```

=== OCI DATA TO COLLECT ===

1. COMPUTE INSTANCES (via `oci.core.ComputeClient` + `MonitoringClient`):
 - Instance OCID, display name, shape, region, compartment, lifecycle state
 - CPU utilization: avg, p50, p95, p99 over the last 15 days (5-min granularity)
 - Memory utilization: avg, p50, p95, p99 (where agent data is available)
 - Network I/O: bytes in/out avg and peak
 - Disk I/O: read/write ops and throughput
 - All metrics via `oci.monitoring.MonitoringClient.summarize_metrics_data()`
2. COST & USAGE (via `oci.usage_api.UsageApiClient`):
 - Cost per instance for the period using `request_summarized_usages()`
 - Cost breakdown: compute, storage, networking per resource
 - Currency, service name, SKU description
 - Group by: `resourceId`, `service`, `compartmentId`

3. BLOCK VOLUMES:
 - Volume size (GB), VPU setting, attached instance OCID
 - Read/write throughput and IOPS averages
4. CONFIG (config/config.yaml) must support:
 - oci_config_path, oci_profile
 - compartment_ocid (list – support multiple)
 - regions (list)
 - collection_period_days: 15
 - metrics_interval_minutes: 60
 - thresholds:
 - cpu_underutilized_pct: 20
 - memory_underutilized_pct: 30
 - cpu_overcommit_pct: 85
 - report:
 - output_dir: ./reports
 - retention_days: 90
 - email:
 - smtp_host, smtp_port, smtp_user, smtp_password (from env)
 - from_address, to_addresses (list)
 - use_tls: true
 - schedule:
 - interval_days: 15

=== IMPLEMENTATION REQUIREMENTS ===

- Use oci-python-sdk, pyyaml, python-dotenv, pandas, structlog
- OCI auth must support both config-file auth and instance-principal auth (detect automatically, fall back gracefully)
- All API calls must have retry logic: exponential backoff, max 3 retries, handle oci.exceptions.ServiceError with status 429 and 5xx
- Paginate all list/summarize calls – never assume single-page responses
- Collect data for ALL instances across ALL configured compartments and regions
- Store raw collected data as typed dataclasses (defined in models/schemas.py)
- The collector must be callable independently: `python main.py collect` and must write raw JSON to reports/raw/<timestamp>_raw.json for audit trail
- logger.py: use structlog, output JSON in production, colored human-readable in development (detect via LOG_FORMAT env var)
- All sensitive values (SMTP password, OCI auth) must be loaded from env vars via python-dotenv; never from config.yaml directly
- Add a `validate\_config()` function that checks all required fields on startup and fails fast with a clear error message listing what is missing

=== TESTING ===

- Write unit tests in tests/test_collector/test_oci_client.py
- Mock all OCI SDK calls using unittest.mock.patch
- Test: successful auth, retry on 429, pagination across 2 pages, empty compartment response, missing metrics gracefully handled

=== COMPATIBILITY ===

- Python 3.9+ only
- Must run on Ubuntu 22.04, RHEL 8, and Windows Server 2022 / Windows 11
- Use pathlib.Path throughout – never os.path string concatenation
- requirements.txt must pin major versions

Output all files with full content. Do not use placeholder comments.
 After creating all files, run: `python main.py collect --dry-run`
 and show the log output to confirm the scaffold works end-to-end.

PHASE 2

Analytics engine

Statistical rigor — patterns, confidence, anomalies

You are a senior data engineer with deep experience in cloud FinOps and statistical analysis. Phase 1 is complete: the collector writes raw JSON to reports/raw/<timestamp>_raw.json containing instances, metrics (5-min granularity over 15 days), cost records, and volumes.

Your task: build src/analytics/ – a rigorous, defensible analytics engine whose recommendations a CFO would trust.

=== ARCHITECTURE ===

```
src/analytics/
■■■ __init__.py
■■■ engine.py # orchestrator with dependency-injected analyzers
■■■ loader.py # raw JSON -> typed pandas DataFrames with schema validation
■■■ utilization.py # statistical utilization profiling
■■■ cost_mapper.py # cost-to-resource attribution with unallocated handling
■■■ right_sizer.py # shape recommendation engine
■■■ ratios.py # fleet-level KPIs
■■■ anomaly.py # multi-signal outlier detection
■■■ confidence.py # confidence scoring framework (shared utility)
■■■ shape_catalog.py # OCI shape catalog with pricing lookup
```

config/shapes.json – OCI shape catalog (see SHAPE CATALOG section below)

=== DATA LOADING & VALIDATION (loader.py) ===

The collector's output cannot be trusted blindly. loader.py must:

- Define pandera (or pydantic) schemas for: InstanceRecord, MetricPoint, CostRecord
- Validate every record on load; collect violations into a ValidationReport
- Reject the run with a clear error if >5% of records fail validation
- Handle these real-world anomalies gracefully:
 - * Metrics with gaps (instance was stopped) – fill with NaN, do not interpolate
 - * Cost records pointing to deleted/terminated resources – bucket as "orphaned_cost"
 - * Multi-currency cost records – convert to a single base currency using a fx_rates section in config (fail fast if a rate is missing)
 - * Duplicate metric points (OCI sometimes returns these) – dedupe on (resource_id, metric_name, timestamp), keeping the latest
 - * Instances with <24h of metric history (recently created) – flag as "insufficient_data" and exclude from right-sizing but include in cost reporting

Output: tuple of (instances_df, metrics_df, costs_df, validation_report)

=== UTILIZATION PROFILING (utilization.py) ===

Per instance, compute the FULL distribution – not just averages. Averages lie.

REQUIRED METRICS:

```
cpu_mean, cpu_median, cpu_p25, cpu_p75, cpu_p90, cpu_p95, cpu_p99, cpu_max
memory_* (same set, where agent data exists)
network_in_*, network_out_* (same set)
disk_read_iops_*, disk_write_iops_*
```

UTILIZATION PATTERN CLASSIFICATION:

Classify each instance into one of these patterns (this drives right-sizing logic):

1. STEADY – cpu_p95 - cpu_p25 < 15% (flat utilization)
2. BURSTY – cpu_p99 / cpu_median > 3 (occasional spikes)

3. CYCLICAL – autocorrelation at 24h lag > 0.6 (daily pattern)
4. WEEKLY – autocorrelation at 168h lag > 0.6 (workday pattern)
5. TRENDING_UP – linear regression slope > 5%/week (growing load)
6. IDLE – cpu_p95 < 5% AND network_p95 < 1MB/s (zombie candidate)
7. ERRATIC – none of the above

This classification is critical: a BURSTY instance at 15% avg CPU still needs its peak headroom; a STEADY instance at 15% can be downsized aggressively.

COMPOSITE UTILIZATION SCORE:

Use a sigmoid-weighted approach, not a flat weighted average:

```
cpu_score = sigmoid_score(cpu_p95, target=70) # rewards being near 70%
memory_score = sigmoid_score(memory_p95, target=70)
io_score = sigmoid_score(io_utilization_pct, target=60)
```

```
composite = 0.45 * cpu_score + 0.35 * memory_score + 0.20 * io_score
```

where sigmoid_score returns 0 at 0%, 1.0 at the target, and decays gracefully past the target (over-utilization is also a problem)

Document the formula in a docstring with a worked example.

=== COST ATTRIBUTION (cost_mapper.py) ===

Real-world cost data is messy. Handle ALL of these:

- Direct attribution: cost_records joined to instances on resource_id
- Indirect costs: block volumes attached to instances -> attribute to parent instance (sum of attached volume costs added to instance.attached_storage_cost)
- Orphan costs: cost_records with resource_id not in active instances -> bucket as "orphaned" and report separately (these are often forgotten resources still generating bills – high-value finding)
- Shared/untaggable costs: report as "fleet_overhead" (don't dilute per-instance metrics)

PER-INSTANCE COST FIELDS:

```
compute_cost – the instance's own compute charges
attached_storage_cost – cost of all attached block volumes
network_egress_cost – outbound data transfer attributed via metrics
total_cost – sum of the above
daily_cost_avg, daily_cost_stddev (volatility detection)
cost_per_vcpu_hour, cost_per_gb_ram_hour
effective_cost_ratio – composite_utilization_score / total_cost
                        (higher = better value extracted per dollar)
wasted_spend_estimate – total_cost * (1 - composite_utilization_score)
                        with a floor: never report waste on an instance with
                        <7 days of data, never report waste on
                        insufficient_data instances
```

Currency: all values stored as Decimal, rounded to 4dp internally, 2dp for display.

=== RIGHT-SIZING (right_sizer.py) ===

This is where most cost tools fail – they recommend shapes that won't actually fit.

ALGORITHM:

For each instance with sufficient_data == True:

1. Determine REAL resource demand:


```
required_vcpu = ceil(cpu_p95 / 100 * current_vcpu * safety_multiplier)
required_ram_gb = ceil(memory_p95 / 100 * current_ram_gb * safety_multiplier)
```

```

safety_multiplier depends on pattern:
    STEADY -> 1.15
    BURSTY -> 1.40 (peaks need headroom)
    CYCLICAL -> 1.25
    WEEKLY -> 1.25
    TRENDING_UP -> 1.50 (account for growth)
    IDLE -> 0.50 (recommend termination, see below)
    ERRATIC -> 1.35

2. Query shape_catalog for candidate shapes:
    - Same family (Flex stays Flex; don't recommend cross-family without justification)
    - vCPU >= required_vcpu AND ram_gb >= required_ram_gb
    - For Flex shapes: compute optimal flex config (vcpu, memory_gb)
    - Sort by hourly cost ascending

3. Validate the candidate:
    - new_estimated_cost = candidate_hourly * 730 (avg hours/month)
    - estimated_savings = current_monthly_cost - new_estimated_cost
    - Reject if savings < 5% (not worth the change risk)
    - Reject if savings < $20/month (operational toil exceeds benefit)

4. Special cases:
    - IDLE pattern -> recommendation_type = "TERMINATE", with prerequisite:
      "Verify no scheduled traffic in the next 30 days before terminating"
    - cpu_p99 > 90% -> recommendation_type = "UPSIZE_OR_INVESTIGATE"
      (these are at-risk, not over-provisioned)
    - insufficient_data -> recommendation_type = "MONITOR", no shape change
    - Flex shape already optimal -> recommendation_type = "OPTIMAL"

5. CONFIDENCE SCORING (confidence.py — shared module):
    confidence = base_score - penalties

    base_score = 1.0
    - Subtract 0.20 if data_coverage_days < 14
    - Subtract 0.15 if pattern == ERRATIC
    - Subtract 0.10 if memory metrics unavailable
    - Subtract 0.10 if cost data has >7 days of gaps
    - Add 0.10 if pattern == STEADY
    - Cap to [0.0, 1.0]

    Map to label:
    >= 0.80 -> "high"
    >= 0.55 -> "medium"
    < 0.55 -> "low"

OUTPUT — Recommendation dataclass:
    instance_id, instance_name
    recommendation_type # DOWNSIZE | UPSIZE_OR_INVESTIGATE | TERMINATE | MONITOR | OPTIMAL
    current_shape, current_config (vcpu, ram_gb)
    recommended_shape, recommended_config
    current_monthly_cost, estimated_monthly_cost, estimated_monthly_savings
    savings_pct
    confidence_score, confidence_label
    rationale # 1-2 sentences explaining the recommendation
    prerequisites # list of steps to verify before executing
    risk_level # low | medium | high

=== FLEET KPIs (ratios.py) ===

All of these — round to 2dp, format with locale-aware separators in the report layer:

FINANCIAL:
    total_fleet_cost_period

```

```

total_fleet_cost_monthly_run_rate # extrapolate to 30 days
orphaned_resource_cost
total_potential_monthly_savings
savings_opportunity_pct
net_recoverable_savings # subtract estimated migration toil ($50/instance)

UTILIZATION:
fleet_avg_cpu_utilization (mean of p95s, weighted by cost)
fleet_avg_memory_utilization
weighted_composite_score (weighted by cost — $1 of waste matters more than 1 instance)

DISTRIBUTION:
overprovisioned_count (composite_score < 0.30)
rightsized_count (0.30 <= composite < 0.70)
underprovisioned_count (composite >= 0.70 OR cpu_p99 > 90%)
idle_count (pattern == IDLE)
insufficient_data_count

EFFICIENCY:
cost_efficiency_index = sum(composite_score * cost) / sum(cost)
                        (0.0-1.0, fleet's overall money-for-utilization ratio)
top_5_wasteful_instances (by wasted_spend_estimate)
top_5_efficient_instances (by effective_cost_ratio)

TREND (requires comparison with previous run if available):
fleet_cost_trend_pct (vs previous 15-day period)
utilization_trend_pct
Flag a "previous run not available – trend metrics suppressed" if first run

=== ANOMALY DETECTION (anomaly.py) ===

Multi-signal – each signal independent, then combined:

SIGNAL 1: Cost outliers
z_score on log(total_cost) across the fleet (log because cost is right-skewed)
Flag |z| > 2.5 as "cost_outlier"

SIGNAL 2: Efficiency outliers
z_score on cost_per_utilization_point
Flag |z| > 2.5 as "efficiency_outlier"

SIGNAL 3: Zombie detection
cpu_p95 < 5% AND network_in_p95 < 100 KB/s AND disk_iops_p95 < 5
for >80% of the period -> "zombie"

SIGNAL 4: Sudden cost spikes
Day-over-day cost increase > 50% in any 24h window -> "cost_spike"

SIGNAL 5: Stranded storage
Block volume with no instance attachment for >7 days -> "stranded_volume"

SIGNAL 6: Reservation mismatch
(if reservation/commitment data is available)
Reserved capacity > p95 demand by >40% -> "over_committed"

Each anomaly: severity = critical | warning | info, suggested_action,
estimated_recoverable_amount.

=== ENGINE ORCHESTRATION (engine.py) ===

class AnalyticsEngine:
    def __init__(self, config: Config, ...injected_analyzers):
        # dependency injection so tests can swap analyzers

```

```
def run(self, raw_data_path: Path) -> AnalyticsResult:
    # 1. load and validate
    # 2. profile utilization for each instance (parallelizable with ProcessPoolExecutor
    # if fleet > 100 instances)
    # 3. attribute costs
    # 4. generate right-sizing recommendations
    # 5. compute fleet KPIs
    # 6. detect anomalies
    # 7. compose AnalyticsResult
```

AnalyticsResult must serialize to JSON via a custom encoder that handles Decimal, datetime, and Enum types. Always include a schema_version field ("1.0.0") so report templates can adapt to future changes.

=== TESTING (REQUIRED, NON-NEGOTIABLE) ===

Use property-based testing where statistical formulas are involved.

```
tests/test_analytics/
■■■ conftest.py # fixtures: synthetic instance generators
■■■ test_loader.py
■■■ test_utilization.py
■■■ test_cost_mapper.py
■■■ test_right_sizer.py
■■■ test_ratios.py
■■■ test_anomaly.py
■■■ test_engine_integration.py
```

REQUIRED TEST CASES:

test_loader.py:

- Valid raw data -> 0 violations
- 10% of records missing required fields -> run rejected with clear error
- Multi-currency without fx_rates -> fails fast with the missing currency name
- Duplicate metric points -> deduplicated, latest kept
- Cost record for terminated instance -> bucketed as orphaned

test_utilization.py:

- Synthesize a STEADY instance (constant 50% CPU); assert pattern detected, cpu_p25 ~ cpu_p95
- Synthesize a BURSTY instance (mostly 10%, occasional 90% spikes); assert pattern, p99 >> median
- Synthesize a CYCLICAL instance (sin wave with 24h period); assert pattern
- Property test (hypothesis): for any composite_score input, output in [0.0, 1.0]
- Edge: instance with 0% utilization -> composite == 0, no division-by-zero
- Edge: 100% utilization -> composite < 1.0 (over-utilization penalized by sigmoid)

test_cost_mapper.py:

- Instance with attached volumes -> attached_storage_cost == sum of volume costs
- Orphaned cost -> reported in fleet_kpis.orphaned_resource_cost, not per-instance
- Multi-currency conversion -> uses configured fx rate, raises if rate missing
- Instance with no cost records -> total_cost == 0, no_billing_data flag set

test_right_sizer.py:

- STEADY 15% CPU instance -> DOWNSIZE recommendation, savings > 0, confidence high
- BURSTY 15% avg / 85% p99 instance -> DOWNSIZE rejected OR very modest, low confidence
- IDLE instance -> TERMINATE recommendation with prerequisites
- 95% p99 CPU instance -> UPSIZE_OR_INVESTIGATE
- Instance with 5 days of data -> MONITOR, no change
- Property test: recommended shape always has vcpu >= required_vcpu (fits the load)
- Property test: estimated_savings >= 0 OR rec_type != DOWNSIZE
- Edge: cheapest shape already in use -> OPTIMAL recommendation

test_ratios.py:

- Synthesize fleet of 20 instances -> assert sum identities (sum of per-instance costs == total_fleet_cost, etc.)
- Empty fleet -> all KPIs return defined values (0.0 or None), no crashes
- First run (no previous) -> trend KPIs are None with explanation flag

test_anomaly.py:

- Inject one zombie + one cost outlier + one stranded volume -> all three flagged, no false positives on healthy instances
- Property test: z-score detection symmetric (positive and negative outliers)

test_engine_integration.py:

- Full pipeline on a 30-instance synthetic fleet
- Assert: report serializes to valid JSON, deserializes back to equivalent object, run completes in <5s

Coverage target: >= 90% on src/analytics/. Enforce in CI.

=== SHAPE CATALOG ===

config/shapes.json must include AT MINIMUM:

- VM.Standard.E4.Flex (1-64 vCPU, 1-1024 GB RAM, AMD, \$0.025/OCPU/hr)
- VM.Standard.E5.Flex (latest AMD)
- VM.Standard3.Flex (Intel Ice Lake)
- VM.Standard.A1.Flex (Ampere ARM, often cheapest)
- VM.Optimized3.Flex (high freq Intel)
- VM.DenseIO.E4.Flex (NVMe local)
- BM.Standard.E4.128 (bare metal, fixed)
- BM.Standard.A1.160
- VM.GPU.A10.1, VM.GPU.A10.2

For each shape:

shape_name, family, processor, min_vcpu, max_vcpu, min_ram_gb, max_ram_gb, is_flex (bool), hourly_cost_per_ocpu, hourly_cost_per_gb_ram, network_gbps, max_block_volume_iops

PRICING DISCLAIMER: prices are list prices; document in the README that contracted/UCM rates may differ. Add a config option `pricing\_override\_path` that lets users supply their actual contract pricing.

=== CLI ===

```
`python main.py analyze [--input <path>] [--output <path>]`
- Defaults to latest raw file
- --validate-only: run schema validation, print report, exit
- --explain <instance_id>: print detailed reasoning for one instance's recommendation (for debugging and CFO questions)
```

=== DELIVERABLES ===

After implementation, run and show output for:

1. pytest tests/test_analytics/ -v --cov=src/analytics --cov-report=term-missing
2. python main.py analyze --input tests/fixtures/scenario_mixed_raw.json
3. python main.py analyze --explain <some_instance_id_from_fixture>

The --explain output should be human-readable and include:

- Why this pattern was detected (with the actual stats)
- Why this confidence level (which penalties applied)
- Why this shape was chosen (showing 2 alternatives that were rejected and why)

PHASE 3

PDF report generator

Executive-grade reporting — narrative, charts, accessibility

You are a senior Python engineer specialising in executive-grade reporting. Phase 2 is complete: `AnalyticsResult` is a fully-typed object with fleet KPIs, per-instance analyses, recommendations, and anomalies.

Your task: build `src/reporter/` — a report so polished an executive forwards it to the board without editing.

=== DESIGN PRINCIPLES ===

1. Every chart must answer a SPECIFIC business question. No decorative charts.
2. Every page must work standalone (someone reading only one page understands it).
3. Numbers always have units, currency symbols, and locale-correct separators.
4. Every recommendation links to its rationale.
5. Print-ready: A4 and Letter, 300 DPI charts, CMYK-safe colors.

=== ARCHITECTURE ===

```
src/reporter/
■■■ __init__.py
■■■ builder.py
■■■ styles.py # ALL visual constants in one place
■■■ components/ # reusable building blocks
■ ■■■ kpi_card.py
■ ■■■ status_badge.py
■ ■■■ data_table.py
■ ■■■ callout_box.py
■ ■■■ footer.py
■■■ charts/
■ ■■■ __init__.py
■ ■■■ base.py # shared chart styling, font registration, DPI
■ ■■■ distribution.py
■ ■■■ scatter.py
■ ■■■ waterfall.py
■ ■■■ trend.py
■ ■■■ heatmap.py
■ ■■■ treemap.py
■■■ sections/
■ ■■■ cover.py
■ ■■■ executive_summary.py
■ ■■■ fleet_overview.py
■ ■■■ cost_analysis.py
■ ■■■ utilization_analysis.py
■ ■■■ recommendations.py
■ ■■■ anomalies.py
■ ■■■ instance_detail.py
■ ■■■ methodology.py
■■■ narrative.py # auto-generates English narrative from KPIs
```

=== STYLES (styles.py — single source of truth) ===

```
# Color system — semantic, not decorative
PRIMARY = '#0F2D52' # navy — headers, titles
PRIMARY_LIGHT = '#3B5F8C'
ACCENT = '#00838F' # teal — KPI highlights
SUCCESS = '#2E7D32' # darker greens read better in print
WARNING = '#EF6C00'
DANGER = '#C62828'
NEUTRAL_900 = '#1A1A1A' # body text
```

```

NEUTRAL_700 = '#4A4A4A' # secondary text
NEUTRAL_500 = '#9A9A9A'
NEUTRAL_200 = '#E5E5E5' # rule lines
NEUTRAL_50 = '#F8F8F8' # zebra rows

# Typography – register once, reuse everywhere
# Use Helvetica (built-in to ReportLab) but expose a config option to use
# Inter or a custom corporate font if the user supplies a TTF path

H1 = ParagraphStyle('h1', fontSize=22, leading=26, textColor=PRIMARY,
                    fontName='Helvetica-Bold', spaceAfter=12)
H2 = ParagraphStyle('h2', fontSize=16, leading=20, textColor=PRIMARY,
                    fontName='Helvetica-Bold', spaceAfter=8)
H3 = ParagraphStyle('h3', fontSize=12, leading=16, textColor=NEUTRAL_900,
                    fontName='Helvetica-Bold', spaceAfter=6)
BODY = ParagraphStyle('body', fontSize=10, leading=14, textColor=NEUTRAL_900,
                    fontName='Helvetica')
CAPTION = ParagraphStyle('caption', fontSize=8, leading=10,
                        textColor=NEUTRAL_500, fontName='Helvetica-Oblique')

# Layout
PAGE_SIZE = A4 # configurable
MARGIN = 0.6 * inch
GUTTER = 0.25 * inch

=== CHART REQUIREMENTS (charts/base.py) ===

Every chart inherits from a base class that enforces:
- Figure DPI = 200 (300 for print runs)
- Font family from styles
- No 3D, no chartjunk, no rainbow palettes
- Always: title (12pt), axis labels with units, source caption (8pt italic at bottom)
- Y-axis: starts at 0 unless explicitly justified (and document why)
- Grid: light horizontal only, no vertical
- Save to BytesIO via savefig(buf, format='png', dpi=DPI, bbox_inches='tight',
    facecolor='white')

All chart functions: chart_xxx(analytics_result, **opts) -> BytesIO

=== PDF STRUCTURE ===

PAGE 1 – COVER
  Top band (full width, 1.5 inch tall, PRIMARY): tool name + report period
  Center: huge headline number – "$X,XXX/month potential savings"
  Below: 3 supporting metrics – "{n} instances analyzed | {n} recommendations |
      {n} anomalies detected"
  Bottom band: organization name (from config), generated_at, run_id (uuid)

PAGE 2 – TABLE OF CONTENTS (auto-generated, with page numbers)

PAGE 3 – EXECUTIVE SUMMARY
  - 4 KPI cards in a 2x2 grid:
    Total Fleet Cost (period) | Potential Monthly Savings | Avg Composite Score |
    Anomaly Count
    Each card: large number, supporting label, trend arrow vs previous period
  - Auto-generated narrative paragraph (3-5 sentences) – see narrative.py below
  - Three callout boxes (color-coded):
    What's working: best-utilized instances, healthy patterns
    What needs attention: top right-sizing wins
    Urgent: critical anomalies, idle resources

PAGE 4 – COST ANALYSIS
  Chart 4.1 – Cost Treemap: each box = one instance, area proportional to total_cost,

```

color = composite_score (red->green). Immediately answers "where is the money going?"
 Chart 4.2 – Daily Cost Trend: line chart, daily fleet cost over the period,
 with anomaly markers and a 7-day moving average

PAGE 5 – UTILIZATION ANALYSIS

Chart 5.1 – Composite Score Distribution: histogram with the four zones
 (over-provisioned / rightsized / underprovisioned / critical) shaded
 Chart 5.2 – Cost vs. Utilization Scatter: x=composite_score, y=monthly_cost,
 bubble size=vcpu_count, quadrant labels:
 "Sweet spot" (high util, moderate cost)
 "Money pit" (low util, high cost) – these are the priority targets
 "Strapped" (high util, modest cost) – risk of throttling
 "Healthy" (low util, low cost)

PAGE 6 – RECOMMENDATIONS (priority order, paginated)

Top of page: summary band – "Total recoverable: \$X | High-confidence: \$Y"
 For each recommendation, a card containing:
 - Status badge (DOWNSIZE/UPSIZE/TERMINATE/MONITOR)
 - Instance name and OCID (truncated)
 - Current -> Recommended (visual arrow)
 - Estimated saving + savings_pct
 - Confidence pill (high/medium/low) with color
 - One-line rationale
 - Prerequisites (collapsible bulleted list)
 - Risk level
 Sort: high-confidence high-savings first
 Group by recommendation_type with section dividers

PAGE 7 – ANOMALIES & ALERTS

Each anomaly as a row in a structured table:
 Severity icon | Instance | Anomaly Type | Description | Suggested Action |
 Recoverable Amount
 If empty: a green callout with a checkmark and "No anomalies detected this period"

PAGE 8 – INSTANCE DETAIL (paginated, 25 rows per page)

Full table of all instances with all key metrics
 Zebra striping (NEUTRAL_50 alternate rows)
 Status column with color-coded badges
 Sort: by wasted_spend_estimate desc

PAGE N – METHODOLOGY

- Data sources and collection period
 - Threshold values used (read from config – make this dynamic)
 - Composite score formula with example calculation
 - Confidence scoring rules
 - Pattern classification rules
 - Limitations:
 "Memory metrics depend on OCI Compute Agent. Instances without the agent
 show N/A and are scored on CPU/IO only – recommendations for these
 instances carry reduced confidence."
 "Pricing is list price unless contract overrides are provided in config."

PAGE N+1 – APPENDIX: Run metadata

Run ID, configuration snapshot (sanitized – no secrets!), version of tool

=== AUTO-NARRATIVE (narrative.py) ===

This is what makes the report feel intelligent. Generate plain-English summaries:

```
def executive_paragraph(result: AnalyticsResult) -> str:
    """
    Produces something like:
```

```
"Across {n} compute instances analyzed over the {period}, the fleet
incurred ${total_cost:,.0f} in compute and storage charges. {x} instances
({pct}%) are operating below {threshold}% composite utilization, representing
an estimated ${savings:,.0f}/month in recoverable spend. Of these,
{high_conf_count} are high-confidence right-sizing opportunities totaling
${high_conf_savings:,.0f}/month. {anomaly_count} anomalies were flagged,
including {zombie_count} idle instances and {stranded_count} stranded
storage volumes."
"""
```

Use templated sentences, but vary based on data:

- If savings == 0: "The fleet is well-optimized; no significant right-sizing opportunities were identified this period."
- If anomalies == 0: omit the anomaly sentence.
- If first run: omit trend comparisons, add "This is the first run; trend metrics will be available from the next report."

Add similar narrative generators for:

- top_recommendations_paragraph (highlight 3 biggest wins)
- anomaly_summary_paragraph
- trend_summary_paragraph (if previous run available)

=== ACCESSIBILITY & PRINT QUALITY ===

- All colors paired with shape/icon (don't rely on color alone – colorblind safe)
- Status communicated by both color AND text (e.g., "Over-provisioned")
- All charts have title, axis labels with units, and a one-line caption
- Page numbers: "Page {n} of {total} | OCI Cost Optimizer | Run {run_id}"
- PDF metadata: Title, Author, Subject, Keywords (set via Canvas.setTitle etc.)
- PDF/A-2b compliance is a stretch goal – at minimum, ensure embedded fonts and no external resource references

=== ERROR HANDLING ===

The reporter must NEVER crash on edge data:

- Empty fleet -> renders a single-page "no data" report with explanation
- Single instance -> all charts and tables still render correctly
- All anomalies / no anomalies -> both render correctly
- Missing memory metrics -> display "N/A" not "NaN" or empty
- Very long instance names -> truncate with ellipsis at 40 chars in tables
- Long rationales -> wrap properly, no overflow

Wrap each section's render in try/except – if a section fails, log the error, insert a placeholder page noting "Section X temporarily unavailable" with the run_id, and continue. The report MUST be deliverable even if one chart breaks.

=== TESTING ===

tests/test_reporter/

```
■■■ conftest.py # mock AnalyticsResult fixtures (3 scenarios)
■■■ test_charts.py
■■■ test_components.py
■■■ test_narrative.py
■■■ test_sections.py
■■■ test_builder.py
■■■ test_visual_regression.py
```

REQUIRED TESTS:

test_charts.py:

- Each chart function: returns BytesIO with PNG header bytes b'\x89PNG'
- Chart with empty data -> returns a "no data" placeholder, doesn't raise
- Chart respects DPI parameter

- Chart fits within max width specified

test_narrative.py:

- All sentence templates produce grammatically correct English
- Edge: \$0 savings -> uses "well-optimized" template, no \$0/month sentence
- Edge: 0 anomalies -> anomaly sentence omitted
- No template variable left unfilled (regex check for {.*})

test_builder.py:

- Healthy fleet (all instances rightsized) -> PDF generates, all sections present
- Wasteful fleet (10 instances over-provisioned) -> PDF generates, recommendations page lists all 10
- Empty fleet -> single-page report with "no data" message
- Single anomaly -> renders correctly
- 100-instance fleet -> instance detail paginates correctly (4 pages of tables)
- PDF output is a valid PDF (open with pypdf, assert page_count >= 5)
- File size is reasonable (between 50 KB and 5 MB for typical reports)

test_visual_regression.py (lightweight):

- Generate the PDF for SCENARIO_MIXED, hash the page count + each section's approximate byte size; assert these don't drift unexpectedly across changes
- Save a sample PDF to tests/golden/sample_report.pdf for manual visual review during PRs

=== CLI ===

```
`python main.py report [--analytics <path>] [--output <path>] [--page-size A4|Letter]`
```

After build, output:

Report generated: reports/OCI_Cost_Report_20250507_120000.pdf

Pages: 12 | Size: 247 KB | Run ID: a1b2c3d4

=== DELIVERABLES ===

After implementation, run and show:

1. `pytest tests/test_reporter/ -v --cov=src/reporter`
2. `python main.py report --analytics tests/fixtures/scenario_mixed_analytics.json`
3. Report file path, file size, and page count
4. Visual review checklist: open the PDF and verify cover, exec summary, and recommendations pages render as designed

PHASE 4

Delivery, scheduling & orchestration

Reliable delivery — SMTP, multi-channel, cross-platform schedule

You are a senior DevOps engineer with deep production reliability experience. Phases 1-3 are complete. The pipeline can collect, analyze, and generate PDFs.

Your task: build the delivery layer (email + alternative channels), the orchestrator, and a cross-platform scheduler that real ops teams trust.

=== ARCHITECTURE ===

```
src/notifier/
■■■ __init__.py
■■■ base.py # NotificationChannel abstract base class
■■■ email_channel.py # SMTP implementation
■■■ slack_channel.py # OPTIONAL — webhook implementation
■■■ teams_channel.py # OPTIONAL — webhook implementation
■■■ templates/
■ ■■■ email_html.j2 # jinja2 HTML template
■ ■■■ email_text.j2 # plaintext fallback
■ ■■■ slack_blocks.j2 # Slack block kit JSON
■ ■■■ teams_card.j2 # Teams adaptive card JSON
■■■ dispatcher.py # routes to enabled channels with fallback
```

```
src/orchestrator/
■■■ __init__.py
■■■ pipeline.py # PipelineRunner class
■■■ steps.py # Step abstraction (idempotent, retryable)
■■■ state.py # run state tracking, recovery
```

```
scripts/
■■■ setup_schedule.py
■■■ install.sh
■■■ install.ps1
■■■ uninstall.sh
■■■ uninstall.ps1
```

=== EMAIL CHANNEL (email_channel.py) ===

Class: SMTPEmailChannel(NotificationChannel)

```
Configuration (config.yaml.email):
  enabled: bool
  smtp_host, smtp_port
  encryption: starttls | ssl | none # explicit, not auto-detected
  auth_method: plain | login | oauth2 | none
  smtp_user (env)
  smtp_password (env)
  smtp_oauth_token (env, if oauth2)
  from_address
  from_name # display name "OCI Cost Optimizer"
  reply_to # optional
  to_addresses: list
  cc_addresses: list (optional)
  bcc_addresses: list (optional)
  attach_pdf: true
  attach_csv: true # also send raw recommendations as CSV (CFO-friendly)
  embed_summary_image: true # KPI summary as inline embedded image
  size_limit_mb: 25 # warn if attachments exceed
  test_mode: bool # if true, write to file instead of sending
```

EMAIL CONTENT REQUIREMENTS:

1. Subject line – dynamic, scannable:
 - "[OCI Cost] {period}: \${savings:,.0f}/mo recoverable across {n} instances"
 - If no savings: "[OCI Cost] {period}: Fleet healthy, {n} instances analyzed"
 - If anomalies: "[OCI Cost] WARN {anomaly_count} anomalies | \${savings:,.0f}/mo recoverable"
2. HTML body – table-based layout (email clients are stuck in 2005):
 - All styles INLINE (no <style> blocks; many clients strip them)
 - Max width 600px
 - Header bar with logo placeholder + report period
 - 4-cell KPI row (matches PDF cover)
 - Top 5 recommendations table
 - Top 3 anomalies (if any)
 - "Full report attached" footer with attachment list
 - Plaintext fallback (multipart/alternative)
 - All numbers formatted with locale-aware separators
3. Headers:
 - Message-ID: deterministic per-run for threading
 - X-Mailer: "OCI-Cost-Optimizer/1.0"
 - X-OCI-Run-ID: <run_id>
 - List-Unsubscribe: configured mailto (if applicable)
 - Date: RFC 2822
 - Auto-generated headers: Auto-Submitted: auto-generated
4. Attachments:
 - PDF (always): "OCI_Cost_Report_{date}.pdf"
 - CSV (optional): "recommendations_{date}.csv" with all rec data
 - JSON (optional, for tool integrators): "summary_{date}.json"
 - Total size <25MB; if larger, attach only PDF and link to local path in the body with explanation
5. Reliability:
 - Connection retry: 3 attempts, 30s linear backoff
 - Per-recipient failure handling: if one recipient fails, log it but continue with the rest; report partial success
 - Use connection pooling: connect once, sendmail to all recipients, quit cleanly
 - Timeout: 60s on connect, 120s on send (configurable)
 - Catch and classify exceptions:
 - SMTPAuthenticationError -> permanent, alert config issue
 - SMTPRecipientsRefused -> log per-recipient, retry on transient codes
 - SMTPServerDisconnected -> retry
 - socket.timeout -> retry
 - SMTPException (generic) -> retry
 - Never log passwords, OAuth tokens, or full email bodies in production
6. Test mode:
 - If EMAIL_TEST_MODE=true OR config.email.test_mode == true:
 - write the rendered email to reports/email_drafts/<run_id>.eml (proper RFC 5322 .eml file, openable in any email client)
 - This must be byte-for-byte identical to what would be sent

=== ALTERNATIVE CHANNELS (optional but architected for) ===

Slack channel (slack_channel.py):

- Posts a Block Kit message via webhook
- Header block + KPI block + top-3 recommendations block + "View full report" button (links to a configurable file-share URL where the PDF can be hosted)
- Configuration: webhook_url (env), channel override, mention list

Teams channel (teams_channel.py):

- Adaptive Card via Incoming Webhook
- Same content shape, MS-Teams-friendly formatting

Both implement the NotificationChannel interface so dispatcher.py can fan out to all enabled channels in parallel.

=== DISPATCHER (dispatcher.py) ===

```
class NotificationDispatcher:
    def __init__(self, config):
        self.channels = [c for c in [SMTPEmailChannel(...), SlackChannel(...), ...]
                           if c.enabled]

    def send(self, run_result: RunResult) -> DispatchResult:
        # Parallel dispatch with ThreadPoolExecutor
        # Per-channel timeout
        # Aggregate per-channel success/failure
        # If ALL channels fail -> escalate (see escalation below)
        # If at least one channel succeeds -> log warnings for failures, return success
```

ESCALATION (when all channels fail):

- Write a critical error file: reports/CRITICAL_DELIVERY_FAILURE_<run_id>.json
- If config.fallback_email_address is set, attempt one last best-effort send to that address using a hardcoded minimal body
- Set non-zero exit code so the scheduler/cron logs it

=== ORCHESTRATOR (orchestrator/pipeline.py) ===

This is what `python main.py run` actually executes.

```
class Step(ABC):
    name: str
    retryable: bool
    timeout_s: int
    @abstractmethod
    def execute(self, context: RunContext) -> StepResult: ...
```

Concrete steps (steps.py):

```
ValidateConfigStep
HealthCheckStep
CollectDataStep
AnalyzeDataStep
GenerateReportStep
DispatchNotificationsStep
CleanupStep # delete reports older than retention_days
```

```
class PipelineRunner:
    def run(self) -> RunResult:
        # Each step runs in order
        # Step failures: log + continue if step.retryable=False AND not critical
        # Critical steps (validate, collect, analyze, report): if any fails,
        # skip generate/dispatch and escalate
        # Notify is special: even if it fails, the report still exists locally
        # Always run cleanup
        # Always write run_log.jsonl entry
        # Always emit RunResult with per-step status
```

RUN STATE & RECOVERY (state.py):

- Each run has a run_id (uuid4) created at start
- Write reports/state/<run_id>.json with current step
- On startup, check for orphaned states (last run didn't complete) and:
 - * If orphaned state is <2 hours old, log warning (run may still be in progress)
 - * If older, attempt resume from last completed step (only if --resume flag)

```

    * Otherwise, mark as failed and start fresh
- Idempotency: if a step's output artifact already exists for this run_id,
  skip and reuse (allows manual reruns of a single step for debugging)

=== CROSS-PLATFORM SCHEDULER (setup_schedule.py) ===

Commands:
python scripts/setup_schedule.py install [--dry-run]
python scripts/setup_schedule.py uninstall
python scripts/setup_schedule.py status
python scripts/setup_schedule.py test # runs the pipeline once via the
                                     configured scheduler invocation

PLATFORM DETECTION:
platform.system() returns: 'Linux', 'Darwin', 'Windows'
Both 'Linux' and 'Darwin' use cron path; 'Windows' uses Task Scheduler.

Linux/macOS — cron:
- Resolve the absolute path to the venv Python: sys.executable
  (or look for ./venv/bin/python relative to the project root if not in venv)
- Resolve the absolute path to main.py
- Resolve LOG_DIR: project_root/logs (create with mode 0755 if missing)
- Cron line:
    0 8 */15 * * cd /abs/path/to/project && /abs/path/to/python main.py run \
    >> /abs/path/to/logs/scheduled_run.log 2>&1
- Detect existing entry by a unique tag comment:
    # OCI_COST_OPTIMIZER_TAG=<deterministic_hash_of_install_path>
- install: read crontab -l, check for tag, replace if present, append if not,
  write back via crontab -
- uninstall: read crontab -l, remove the tagged line, write back
- status: read crontab -l, find tagged line, parse next-run time using
  croniter library (if available) or compute manually
- --dry-run: print the cron line that would be installed, don't modify crontab

WINDOWS — Task Scheduler:
- Generate a Task Scheduler XML definition (more reliable than schtasks args
  which have argument-quoting hell):
  <?xml version="1.0" encoding="UTF-16"?>
  <Task xmlns=...>
    <Triggers><CalendarTrigger>
      <StartBoundary>2025-01-01T08:00:00</StartBoundary>
      <ScheduleByDay><DaysInterval>15</DaysInterval></ScheduleByDay>
    </CalendarTrigger></Triggers>
    <Principals>
      <Principal id="Author">
        <UserId>{current user}</UserId>
        <LogonType>InteractiveToken</LogonType>
        <RunLevel>LeastPrivilege</RunLevel>
      </Principal>
    </Principals>
    <Settings>
      <MultipleInstancesPolicy>IgnoreNew</MultipleInstancesPolicy>
      <DisallowStartIfOnBatteries>false</DisallowStartIfOnBatteries>
      <StartWhenAvailable>true</StartWhenAvailable> <!-- catch up if missed -->
      <ExecutionTimeLimit>PT2H</ExecutionTimeLimit>
    </Settings>
    <Actions Context="Author">
      <Exec>
        <Command>{abs path to python}</Command>
        <Arguments>{abs path to main.py} run</Arguments>
        <WorkingDirectory>{abs path to project}</WorkingDirectory>
      </Exec>
    </Actions>
  </Task>

```

```

    </Task>
- install: schtasks /Create /TN "OCICostOptimizer" /XML <xml_file> /F
- uninstall: schtasks /Delete /TN "OCICostOptimizer" /F
- status: schtasks /Query /TN "OCICostOptimizer" /V /FO LIST
- Document that "Run whether user is logged on or not" requires storing
  credentials and is therefore opt-in via --service flag

PERMISSION HANDLING:
- If user lacks permission to modify crontab/Task Scheduler:
  * Print the EXACT line/XML that they need to add manually
  * Print step-by-step instructions
  * Exit with non-zero code
- On Windows, detect admin shell with ctypes.windll.shell32.IsUserAnAdmin();
  if --service was requested but not admin, exit with instructions

=== INSTALL SCRIPTS ===

scripts/install.sh (Linux/macOS):
#!/usr/bin/env bash
set -euo pipefail
- Detect Python 3.9+ (fail with clear message if missing)
- Create venv at ./venv if not exists (or use existing)
- Activate venv
- pip install --upgrade pip wheel
- pip install -r requirements.txt
- Copy config/.env.example -> config/.env if .env doesn't exist (warn user to edit)
- Copy config/config.yaml.example -> config/config.yaml if not exists
- Run: python main.py validate-config (fail loudly if invalid)
- Run: python main.py health (warn if any check fails, but don't abort install)
- Prompt: "Install scheduled task to run every 15 days? [y/N]"
  if yes: python scripts/setup_schedule.py install
- Print summary: where things live, how to run manually, how to view logs
- Idempotent: safe to re-run

scripts/install.ps1 (Windows):
- PowerShell 5.1+ compatible
- Same flow as install.sh, using PowerShell idioms
- Use Set-StrictMode -Version Latest and $ErrorActionPreference = 'Stop'
- Detect Python: prefer py launcher (py -3 --version), fall back to python --version
- Same prompts and output

scripts/uninstall.sh / uninstall.ps1:
- Remove scheduled task / cron entry
- Optionally (with --purge flag): remove venv and reports/
- Never auto-delete config.yaml or .env (user data)

=== CLI ENHANCEMENTS (main.py) ===

`python main.py run [--dry-run] [--skip-notify] [--resume <run_id>]`
--dry-run: execute everything but write to /tmp and use email test_mode
--skip-notify: generate report but don't send (useful for debugging)
--resume: resume a failed run from its last completed step

`python main.py status`
- Show last 5 runs from run_log.jsonl with status
- Show next scheduled run time (parse from cron/Task Scheduler)

`python main.py logs [--tail N] [--follow]`
- Tail the most recent log file
- --follow: live tail (cross-platform polling implementation)

=== TESTING ===

```

```
tests/test_notifier/
■■■ test_email_channel.py
■■■ test_dispatcher.py
■■■ test_templates.py
```

```
tests/test_orchestrator/
■■■ test_pipeline.py
■■■ test_state.py
```

```
tests/test_scheduler/
■■■ test_setup_schedule_linux.py
■■■ test_setup_schedule_windows.py
■■■ test_install_scripts.py
```

REQUIRED TESTS:

test_email_channel.py:

- Mock smtplib.SMTP and SMTP_SSL; verify:
 - * STARTTLS path: connect -> ehlo -> starttls -> ehlo -> login -> sendmail -> quit
 - * SSL path uses SMTP_SSL
 - * Auth=none skips login
- Test mode writes a valid .eml file (parse it back with email.parser.Parser)
- Per-recipient failure: 1 of 3 refused, other 2 succeed -> returns partial_success
- Connection refused -> all 3 retries attempted, then permanent failure
- Attachment >25MB -> warning logged, but email still sent (with PDF only)
- HTML and plaintext both present in multipart/alternative
- All numbers in body match analytics result (regex check for currency formatting)

test_dispatcher.py:

- All channels disabled -> no-op, returns success with empty result
- Email succeeds, Slack fails -> returns partial success, logs Slack error
- All channels fail -> escalation triggered, fallback email attempted
- Parallel dispatch: mock channels with sleep(0.5) each, total elapsed < 1s

test_templates.py:

- jinja2 templates render with all required variables present
- No unresolved {{ }} placeholders in output
- Locale-aware number formatting works for at least en_US and en_IN
- Edge: 0 anomalies, 0 recommendations -> templates degrade gracefully

test_pipeline.py:

- Full happy path: all 6 steps succeed, run_log entry has all green
- Step 3 fails: steps 4-5 skipped, step 6 (cleanup) still runs
- --skip-notify: step 5 marked as "skipped"
- Resume: simulate failure after step 3, resume run, assert steps 1-3 reuse artifacts and step 4 starts fresh
- Run timeout enforced

test_setup_schedule_linux.py:

- Mock subprocess.run for `crontab -l` and `crontab -`
- Install with no existing entries: appended cleanly with tag
- Install with existing tagged entry: replaced (not duplicated)
- Install with existing untagged entries: untagged entries preserved
- Uninstall: only tagged entry removed
- Status: detect installed vs not installed
- --dry-run: no actual crontab modification
- Permission denied: clear error message with manual instructions

test_setup_schedule_windows.py:

- Mock subprocess.run for schtasks
- XML is well-formed (parse with xml.etree.ElementTree)
- Install/uninstall/status commands constructed correctly
- Admin detection: --service without admin -> exits with instructions

```
=== DELIVERABLES ===
```

```
After implementation, run and show:
```

1. `pytest tests/ -v --cov=src --cov-report=term-missing`
(assert overall coverage $\geq$ 80%)
2. `python main.py run --dry-run` (full pipeline with test mode)
3. `python scripts/setup_schedule.py install --dry-run`
(show the cron line / XML that would be installed)
4. `python main.py status` (show run history)
5. `cat reports/email_drafts/<latest>.eml` (verify the rendered email)

PHASE 5

Hardening, security, observability & release

Production readiness — security, chaos tests, packaging

You are a senior site-reliability engineer doing the final hardening pass before this tool goes to production at a real company. Phases 1-4 are functionally complete.

This phase is NOT about adding features — it's about making the tool reliable, secure, observable, and maintainable. Treat every section as required.

=== PART A: SECURITY HARDENING ===

1. SECRETS MANAGEMENT

- Audit the entire codebase: no secret may appear in logs, error messages, stack traces, or written-to-disk artifacts (reports, state files, run_log)
- Implement `src/utils/redaction.py`:
`redact_secrets(obj: Any) -> Any # recursive, redacts known sensitive keys`
 Sensitive keys: `password`, `token`, `api_key`, `secret`, `auth`, `credential`,
`smtp_pass*`, `oci_*_key` (any case)
- All log handlers must apply this redactor as a filter
- Test: log a dict containing fake secrets, assert the output contains `***REDACTED***`

2. CREDENTIAL SOURCES (in priority order, document this):

- Environment variables (highest priority)
- OS keychain (optional via ``keyring`` library, behind a config flag)
- `.env` file (development only; warn if used in prod via `PROD_MODE` env var)
- OCI instance principals (when running on OCI compute — preferred for prod)

3. FILE PERMISSIONS

- On install, set:
`config/.env -> 0600`
`config/config.yaml -> 0644`
`reports/ -> 0750`
`logs/ -> 0750`
- Windows equivalent: use `icacls` to restrict to current user
- Refuse to start if `.env` is world-readable (mode & `00077`)

4. INPUT VALIDATION

- All config values: validate with `pydantic BaseSettings`
- All OCID inputs: regex-validate format (`ocid1\.[a-z]+\.[a-z0-9.-]+`)
- All email addresses: validate with `email-validator` library
- SMTP host: must be hostname or IP, not arbitrary URL
- Path inputs: resolve and verify within an allowlist of base directories (prevent path traversal even though this is a single-tenant tool)

5. DEPENDENCY SECURITY

- `requirements.txt`: pin all versions (`==`), not `>=` or `~=`
- Add `requirements-dev.txt` for dev dependencies
- Run ``pip-audit`` (or ``safety check``) in CI; fail on HIGH/CRITICAL CVEs
- Document supply chain: where deps come from, how often to update

6. NETWORK

- All HTTP clients: explicit timeouts (`connect=10s`, `read=60s`)
- SMTP connection: verify TLS certificate by default (config opt to disable for self-signed corp SMTP, but log a WARNING when disabled)
- Refuse to send to non-corporate domains if `config.email.allowed_domains` is set

=== PART B: OBSERVABILITY ===

1. STRUCTURED LOGGING (`src/utils/logger.py` — enhance)

Every log entry includes:
`timestamp (ISO 8601 with timezone)`

```

level
logger_name
run_id (when in a run context)
step_name (when in a step)
instance_id (when processing one)
message
extra (dict of context-specific fields)

```

Two output modes:

- LOG_FORMAT=json (production): one JSON object per line
- LOG_FORMAT=human (development): colored, indented, readable

Log rotation: logs/ rotates daily, keeps last 30 days, total cap 1GB
(use logging.handlers.TimedRotatingFileHandler)

2. METRICS (src/utils/metrics.py)

Emit timing/counter metrics for each step:

```

pipeline.step.duration_seconds{step="collect"}
pipeline.step.outcome{step="analyze",status="success"}
collector.api_calls_total{endpoint="ListInstances"}
collector.retries_total{endpoint="...",status_code="429"}
analytics.instances_processed_total
analytics.recommendations_generated{type="DOWNSIZE"}
reporter.pdf_size_bytes
notifier.emails_sent_total{status="success|partial|failed"}

```

Backends (configurable, pluggable):

- file (default): append metrics to logs/metrics.jsonl
- statsd (optional)
- prometheus pushgateway (optional)
- datadog (optional, via dogstatsd)

3. HEALTH CHECK (src/utils/health.py – enhance)

```
`python main.py health [--verbose] [--json]`
```

Checks (each with status, latency_ms, details):

- oci_credentials_valid
- oci_compartment_accessible (per configured compartment)
- oci_monitoring_api_reachable
- oci_usage_api_reachable
- smtp_connection (test connection only, don't send)
- smtp_auth (only if --auth-test flag, since this can lock accounts)
- filesystem_writable (output_dir, log_dir, state_dir)
- disk_space_available (warn if <1GB free in output_dir)
- python_version_supported
- last_run_status (from run_log.jsonl)
- schedule_installed

Exit codes:

- 0 = all checks passed
- 1 = warning (non-critical issue)
- 2 = critical failure (tool will not function)

--json flag outputs JSON for monitoring system integration

4. RUN HISTORY & TRENDS

- run_log.jsonl: keep last 100 entries (rotate older to run_log_archive.jsonl)
- `python main.py history [--last N] [--format table|json]` shows runs
- The PDF report's trend section reads from run_log to compute deltas

=== PART C: RELIABILITY ===

1. IDEMPOTENCY

- Every step writes to a deterministic path keyed on run_id
- Re-running a step with the same run_id should produce the same output (or skip if output exists)

- Document any non-idempotent operations (e.g., sending email – use the `run_id` in the Message-ID and as a duplicate-suppression key)

2. ERROR BUDGETS

- Define and document SLOs:
 - Pipeline succeeds end-to-end: 95% of runs (allowing for transient OCI/SMTP issues)
 - Report generated: 99% (must work even if notify fails)
 - Time to complete: <15 minutes for fleets up to 500 instances
- On 3 consecutive failures, send an alert email to a configured ops address with the last 3 `run_log` entries

3. TIMEOUTS EVERYWHERE

- OCI API calls: 60s
- SMTP send: 120s
- PDF generation: 5min hard cap
- Whole pipeline: 30min hard cap (configurable)
- On timeout, log clearly which step timed out and what was in flight

4. RESOURCE LIMITS

- Memory: profile with `memory_profiler` on a 500-instance synthetic fleet; ensure peak memory <1GB
- Disk: report retention enforced (default 90 days), oldest deleted on each run
- File descriptors: ensure all SMTP/HTTP connections close cleanly (use context managers everywhere – verify with a leak test in CI)

=== PART D: TEST SUITE COMPLETENESS ===

1. SYNTHETIC FIXTURE FACTORY (`tests/fixtures/factory.py`)

Build a programmatic factory that generates realistic test data:

```
class FleetFactory:
    def healthy(self, n=10) -> RawData
    def wasteful(self, n=10) -> RawData
    def mixed(self, n=15) -> RawData
    def empty(self) -> RawData
    def single(self, **overrides) -> RawData
    def with_anomalies(self, types: list[str]) -> RawData
    def large(self, n=500) -> RawData
```

Each instance generated with:

- realistic OCID, shape (from `shape_catalog`), compartment, region
- metric series with the specified pattern (steady/bursty/cyclical/idle/etc.)
- cost records sized appropriately to the shape
- optional missing data, gaps, multi-currency, etc.

2. INTEGRATION TESTS (`tests/test_integration/`)

`test_full_pipeline_happy.py`:

- Patch OCI SDK, patch SMTP, run `main.py run`, assert all artifacts exist
- Verify `run_log.jsonl` entry has all steps green
- Open the PDF, verify page count and metadata

`test_full_pipeline_degraded.py`:

- OCI returns intermittent 429s -> pipeline succeeds with retries
- Memory metrics missing for half the fleet -> analyses still produced, confidence reduced, report renders
- SMTP fails permanently -> report generated locally, `run_log` shows partial-success

`test_full_pipeline_resume.py`:

- Inject failure after analyze step
- Re-run with `--resume` -> starts from report step, completes successfully

3. PERFORMANCE TESTS (tests/test_performance/)

```
test_large_fleet.py:
- 500 instances, full pipeline, mocked OCI
- Assert: completes in <10 minutes
- Assert: peak memory <1GB
- Assert: PDF size <10MB
- Assert: no test artifacts left in /tmp after teardown
```

4. CHAOS TESTS (tests/test_chaos/)

```
test_chaos_oci.py:
- Random 5% of OCI calls return 5xx
- 10% return malformed JSON
- 2% time out
- Pipeline still completes; failed instances bucketed correctly

test_chaos_filesystem.py:
- Read-only output dir -> fails fast with clear error
- Disk full simulation -> handled gracefully (catch OSError)
```

5. CROSS-PLATFORM TESTS

```
GitHub Actions matrix:
  OS: ubuntu-latest, macos-latest, windows-latest
  Python: 3.9, 3.10, 3.11, 3.12
All tests must pass on all 12 combinations.
```

6. COVERAGE

```
pytest --cov=src --cov-report=term-missing --cov-fail-under=85
Per-module minimums:
  src/analytics/: 90%
  src/reporter/: 80%
  src/notifier/: 85%
  src/orchestrator/: 90%
  src/utils/: 80%
```

=== PART E: PACKAGING & DISTRIBUTION ===

1. PYTHON PACKAGE

```
pyproject.toml fully configured for:
- build (setuptools or hatch)
- entry_points: oci-cost-optimizer = main:cli
- extras_require: [slack], [teams], [keyring], [dev]
`pip install .` installs as a CLI tool
Version stored in src/__version__.py, read once
```

2. DOCKER

```
Multi-stage Dockerfile:
  Stage 1 (builder): python:3.11-slim, install build deps, pip install --target /app/deps
  Stage 2 (runtime): python:3.11-slim, copy /app/deps, copy src/, non-root user
  Final image <300MB, scanned with `trivy` in CI
docker-compose.yml with:
- Service definition with env_file, volumes for config/ and reports/
- Optional sidecar: ofelia for cron-style scheduling inside compose
- Optional sidecar: a simple nginx serving reports/ for internal access
Health check using `python main.py health`
```

3. KUBERNETES MANIFESTS (optional but architected)

```
k8s/cronjob.yaml: schedule: "0 8 */15 * *", parallelism: 1, backoffLimit: 2,
                  successfulJobsHistoryLimit: 5, failedJobsHistoryLimit: 5
k8s/configmap.yaml: config.yaml mounted into /app/config
k8s/secret.yaml: SMTP credentials, OCI credentials (with documentation that
                 these should come from a real secrets manager in prod)
```

k8s/pvc.yaml: persistent volume for reports/

4. RELEASE

- GitHub Actions: on tag v*, build wheel + sdist, build Docker image, push to a registry, create GitHub release with changelog
- CHANGELOG.md following Keep a Changelog format
- SemVer versioning policy documented

=== PART F: DOCUMENTATION ===

1. README.md – restructure into:

- Hero section: 3-line elevator pitch, key features, screenshot
- Quick start (5 commands)
- Architecture overview (with the ASCII diagram)
- Configuration reference (full table of every config key)
- Operations guide (logs, health, troubleshooting)
- Security model
- Contributing
- License

2. docs/ARCHITECTURE.md

- Component diagram
- Data flow: OCI -> collector -> loader -> analytics -> reporter -> notifier
- Sequence diagram for a typical run
- Decision log: why ReportLab over WeasyPrint, why pandas, why Decimal, etc.

3. docs/RUNBOOK.md

- "What to do when the email doesn't arrive"
- "What to do when OCI auth fails"
- "What to do when a run takes longer than usual"
- "How to onboard a new compartment"
- "How to update the shape catalog when OCI releases new shapes"
- "How to update pricing"
- Each section: symptom -> diagnosis steps -> resolution

4. docs/SECURITY.md

- Threat model (what we protect against, what we don't)
- Secrets handling
- Vulnerability disclosure policy
- Dependency update cadence

5. docs/CONFIGURATION.md

- Every config option, default value, type, description, example

6. CHANGELOG.md and CONTRIBUTING.md as standard files

=== PART G: FINAL VALIDATION ===

Run and show output for ALL of the following:

1. Linting:

```
ruff check src/ tests/
ruff format --check src/ tests/
mypy src/ --strict
pylint src/ --rcfile=.pylintrc
```

All must pass with zero errors.

2. Security scans:

```
bandit -r src/
pip-audit --strict
detect-secrets scan --baseline .secrets.baseline
```

```

3. Test suite:
  pytest tests/ -v --cov=src --cov-fail-under=85 --tb=short
  pytest tests/test_performance/ -v --benchmark
  pytest tests/test_chaos/ -v

4. Docker build:
  docker build -t oci-cost-optimizer:dev .
  docker run --rm oci-cost-optimizer:dev health
  docker images oci-cost-optimizer:dev (verify size <300MB)

5. End-to-end smoke test on Linux:
  ./scripts/install.sh
  source venv/bin/activate
  python main.py health
  python main.py run --dry-run
  python scripts/setup_schedule.py install --dry-run
  python scripts/setup_schedule.py status

6. End-to-end smoke test on Windows (PowerShell):
  .\scripts\install.ps1
  .\venv\Scripts\Activate.ps1
  python main.py health
  python main.py run --dry-run
  python scripts/setup_schedule.py install --dry-run
  python scripts/setup_schedule.py status

7. Sample report walkthrough:
  - Generate the PDF for SCENARIO_MIXED
  - Show: file path, file size, page count, metadata (Title, Author, Subject)
  - Open visually and confirm: cover, exec summary, all charts render,
    recommendations sorted correctly, anomalies present, methodology accurate

=== EXIT CRITERIA ===

This phase is COMPLETE when ALL of the following are true:
- All 7 validation steps above pass
- Coverage >= 85% overall, >= 90% on analytics + orchestrator
- Zero HIGH/CRITICAL CVEs in dependencies
- Cross-platform CI (3 OS x 4 Python versions = 12 jobs) all green
- Docker image builds and runs health check successfully
- Documentation is complete and reviewed
- Sample PDF report has been visually inspected and approved

If ANY of these fails, the phase is NOT complete – fix and re-validate.

```

Review checklist & next steps

Decisions to confirm before kickoff

Topic	Default in this brief	Decision needed?
Compartments to cover	All listed in config	Confirm initial list
Regions	All configured	Confirm initial list
Schedule cadence	Every 15 days at 08:00 local	Confirm cadence + time
Email recipients	Configurable list	Provide distribution list
SMTP server	Corporate SMTP via STARTTLS	Provide host + auth
Pricing source	OCI list price (defaults)	Provide contract overrides if any
Underutilization threshold	CPU/memory p95 < 30%	Confirm or adjust
Min savings to recommend	\$20/month or 5%	Confirm or adjust
Slack/Teams delivery	Off by default	Enable if required
Run host	On-prem VM or OCI compute	Confirm; affects auth model

Risks & mitigations

OCI MEMORY METRICS MAY BE SPARSE

Memory metrics depend on the OCI Compute Agent. Where unavailable, the analytics engine marks the value as N/A and reduces confidence on right-sizing recommendations rather than fabricating a number.

PRICING DRIFT

OCI prices change. The shape catalog supports a contract-pricing override and the methodology page in the report explicitly documents the pricing source so recommendations are auditable.

SMTP DELIVERY FAILURES

Multi-attempt retries, per-recipient failure handling, and a fallback address are built in. On total failure, the PDF still exists locally and the run is logged as partial-success rather than lost.

OCI API RATE LIMITS

All API calls use exponential backoff on 429 and 5xx responses with a cap of three retries; pagination is enforced everywhere.

RECOMMENDATIONS ACTED ON WITHOUT REVIEW

Every recommendation includes prerequisites and a risk level. The brief is explicit: this tool recommends, humans execute.

Suggested next steps

- Walk through this document with the engineering team and capture any objections.
- Confirm the decisions table above and lock the initial config values.
- Provision an SMTP service account and OCI credentials with read-only access to Monitoring and Usage APIs.
- Begin Phase 1 in a fresh Claude Code session; review the produced scaffold before kicking off Phase 2.
- After Phase 3, do a visual review of the sample PDF report on real (or near-real) data before signing off.
- Schedule a 30-minute review at the end of each phase to ratify outputs and approve progression.

DEFINITION OF DONE

The project is considered complete when the full pipeline runs end-to-end on a real OCI compartment, generates a visually-reviewed PDF, delivers it via SMTP to the distribution list on schedule, and survives a chaos test with simulated OCI and SMTP failures without losing the report. Coverage must be $\geq 85\%$ overall and $\geq 90\%$ on analytics and orchestrator modules.